

The Info Pharma Ordering System (IPOS)

Summary

This is a Requirements Specification and High-Level Design Document, a proposal to the managing director of Info Pharma Ltd, by team 4, Designers of IPOS-SA subsystem in group []

Authors

[add surnames]

Herman

Joanna

Leon

Kanzah

Vlad

Yamin

Jenis

Distribution

The Managing Director of Info Pharma Ltd, the development team at the Centre for Software Reliability at the School of Informatics, City University London

Contents

1 Preface.....	3
1.1 Purpose and Scope of the document (what it is and what is not).....	3
1.2 Intended audience and Reading Suggestions.....	3
1.3 History of the document.....	3
2 Introduction.....	4
3 Glossary.....	4
4 User Requirements.....	4
5 System requirements.....	4
5.1 Purpose and scope of the system.....	4
5.2 System models.....	4
5.2.1 User stories.....	4
6 Anticipated Evolution of System Requirements.....	4
6.1 Validation.....	4
6.2 Loyalty Cards.....	4
7 Info Pharma High-Level Design.....	4
7.1 IPOS-SA.....	4
7.2 IPOS-CA.....	4
7.3 IPOS-PU.....	4
7.4 Software Architecture.....	4
7.5 Source Code (Python) for the interfaces of IPOS-SA.....	4
7.6 Testing the communications between subsystems.....	5
8 Appendices.....	5
8.1 Initial Statement of requirements by the client.....	5
IPOS: Info Pharma Ordering System.....	5
8.2 Hardware (minimal and optimal configuration).....	5
8.2.1 minimal hardware configuration.....	5
8.2.2 optimal hardware configuration.....	5
9 Index.....	5

1 Preface

1.1 Purpose and Scope of the document

This is a Requirements Specification for one of the subsystems of IPOS, IPOS-SA, and the High-Level Design document for the IPOS software, which includes IPOS-SA and two other subsystems, IPOS-CA and IPOS-PU. The document covers the functional requirement of IPOS-SA and the high-level design of the entire IPOS system showing the IPOS architecture composed of three subsystems and the mechanisms of interaction between them. The document is provided in response to the initial statement of requirements by Info Pharma Ltd, the client.

The document explains what the subsystem shall do, and how its subsystems might interact with one another, but does not cover details about how the functionality of IPOS-SA subsystem will be implemented. Various UML models are used together with fragments of software code to convey in more detail to system developers the expected behaviour of the IPOS-SA subsystems and especially how this subsystem will interact with the other subsystems of IPOS.

1.2 Intended audience and Reading Suggestions

The Managing Director of Info Pharma Ltd, the system analysts and architects at the Centre of Software Reliability (CSR), City University London - smth to discuss is this what we are supposed to put as well?

Some of the sections are intended for the CSR team only, which is clearly stated in the document.

1.3 History of the document

This v1.0 of the documents.

2 Introduction

IPOS will allow Info Pharma Ltd to automate their business process in three interrelated areas:

i) add here, ii) add here and iii) add here. By introducing an online ordering system the merchants will be able to make pharmaceutical orders and subsequently sell these products in their shops. The – add here –

The three areas of planned automation are expected to be covered by three subsystems: IPOS-SA, IPOS-CA and IPOS-PU, respectively. This document is focused on IPOS-SA. The other subsystems are covered in separate documents.

In addition to the services offered to merchants, IPOS-SA allows -add here-

IPOS-SA will also send -add here-

IPOS-SA is integrated with third party services - [add here]. It will interact with the other subsystems, IPOS-CA and IPOS-PU, too.

3 Glossary

Actor	A human or an external system interacting with IPOS e.g. merchant, administrator, manager, etc
CSR	Centre for software reliability
IPOS	Info Pharma Ordering System
UML	Unified Modelling language
User Story	

4 User Requirements

This section is left intentionally blank

5 System requirements

5.1 Purpose and scope of the system

This section provides a more detailed specification of the required functionality (known as system requirements) for IPOS-SA.

The intended audience are the *system architectures, designers and software developers* responsible for the implementation of IPOS. Since we use UML, which requires a degree of specific expert knowledge, we do not expect this section to be immediately useful to the Info Pharma staff but would like to encourage them to review and comment, if possible.

5.2 System models

5.2.1 User stories

The user stories for the IPOS-SA subsystem are shown below in table format.

5.2.1.1 IPOS-SA-ACC

[what this package contains bit]

Title: merchant filling in details for account	Priority: essential	User Story ID: SA-ACC-ME01
User Story: As a merchant, I want to fill in details, So that I can create an account on the system		
Acceptance Criteria: Given the system prompts the merchant to input their account details, When the merchant fills in their details, Then the system should activate an account based on these details the merchant can log into.		

Title: merchant implementing discount plans	Priority: important	User Story ID: SA-ACC-ME02
User Story: As a merchant, I want to implement two types of discount plans, So that my account will be valid to place orders.		

Acceptance Criteria:

Given the system prompts the merchant to input their discount plans,

When they have input at least two plans (one fixed and one flexible),

Then as long as all other details have been added correctly, the subsystem should activate this account as valid and allow orders to be taken.

[for future ref when adding more me = merchant id tag, ma = manager id tag, ad = admin id tag]

5.2.1.2 IPOS-SA-CAT

[what this package contains bit]

5.2.1.3 IPOS-SA-ORD

[what this package contains bit]

5.2.1.4 IPOS-SA-RPT

[what this package contains bit]

5.2.2 Prioritised List of User Stories

The list below prioritises the user stories included in the system requirements according to their importance in IPOS-SA. CSR management will use the assigned priorities to decide on the order of implementing the functionality specified in the user stories according to the resources available: the **essential** user stories will be implemented first, then the user stories ranked as **important** will follow. The user stories ranked as **good-to-have** will be implemented last provided, the resources (manpower and time) are available to complete their implementation.

User Story ID	User Story name	US priority	Justification

6 Anticipated Evolution of System Requirements

We expect that IPOS-SA will be developed and deployed quickly. The operational experience from merchants and staff will inform the future decisions about the necessary changes. The envisaged likely changes to various parts of IPOS-SA are summarised below.

6.1 Validation

Currently the validation of account information provided during registration of their account will be checked by administrators in Info Pharma. In the future, however, we envisage that the validation may be done using 3rd party online services, which will require changes - here we need a group discussion on what may change in the future -

6.2 Loyalty Cards - subject to change

- This section was added as something they may implement in the future we can use the same idea or do smth else [group discussion]

7 Info Pharma High-Level Design

This section defines the high-level design on the entire IPOS software, which includes its architecture - the subsystems which it is made up of, and the communication interfaces between the subsystems. The subsystems envisaged are:

7.1 IPOS-SA

This subsystem will allow merchants to register with Info Pharma to view the catalog of products offered and make orders and discount plans.

7.2 IPOS-CA

This subsystem will allow ...

7.3 IPOS-PU

This subsystem will allow ...

7.4 Software Architecture

The architecture of IPOS as a UML component diagram, developed with Visual Paradigm, Professional Edition, is shown on figure 1.

[paste component diagram with whole system]

Figure 1. IPOS architecture as a UML component diagram. Each subsystem is modelled as a component with stereotype <<subsystem>>. The communication between the subsystems is achieved via a set of assembly connectors (i.e. a pair of “provided” and “required” interfaces referring to the API offered by the respective subsystems. [API names from diagram]) are provided by IPOS-SA, IPOS-CA and IPOS-PU, respectively and the APIs are used to access the functionality offered by one of the subsystems by the other two subsystems.

Figure 1 hides the API details. Using the “presentation options” of Visual Paradigm, one can switch to “class” notations and disclose full interface details, as is illustrated on Figure 2

[paste component diagram with presentation options on]

Figure 2. Now the methods defined in interfaces are explicitly shown with their full signatures. These are plausible public methods via which functionality of respective subsystems can be accessed. Note that changing the presentation of interfaces, changed the relationships of the interfaces too: the subsystems responsible to implement the interface has a “realisation” relationship with it, which implies that the respective subsystem will contain an implementation class realising the interface. The subsystems using the the API (i.e. for which the API is a

“required” interface) now would have a dependency relationship with the interface (<<usage>> icon in Visual Paradigm should be used in the construction of the component diagram).

The IPOS architecture includes three subsystems and shows the interfaces necessary for these subsystems to interact with each other

7.4.1 Design diagram of subsystem IPOS-SA

Next, we elaborate the high-level design of IPOS-SA and provide details on the interfaces that this subsystem needs, and on the implementation of the subsystem’s required interface.

The class diagram of IPOS-SA subsystem is shown in figure 3.

[paste class diagram of IPOS-SA]

Figure 3. High-Level design of IPOS-SA

7.5 Source Code (Python) for the interfaces of IPOS-SA

Now that we have defined all interfaces and the implementation class [nameOfOurAPI], source code could be derived in Python for the interfaces and the implementation classes shown in Figure 3. This can be done with Visual Paradigm. The complete Python code of the interfaces and a skeleton of the implementation class are shown below generated by Visual Paradigm is provided below:

[paste python code into table and then made borders invisible to keep format good]

[method 1]
[method 2]
[etc]

From these snippets of Python code, one can generate [python equivalent of JUnit tests], if necessary for each of them, and develop **test cases** before the code of the implementation classes are available.

7.6 *Testing the communications between subsystems*

Now that we have defined the interfaces necessary for communication between the subsystems of IPOS, and generated Python code for all of them, we develop tests for inter-subsystem communication, i.e. of the respective implementation classes.

[here goes test cases]

Method	Test ID	Parameter values	Comment	Expected outcome

8 Appendices

8.1 Initial Statement of requirements by the client

IPOS: Info Pharma Ordering System

[paste requirements from brief]

8.2 Hardware (minimal and optimal configuration)

8.2.1 minimal hardware configuration

The minimal hardware configuration recommended for IPOS development consists of a desktop PC, 1GB of RAM and at least 50GB of disk space for storing the software programs and data possibly running a database server such as MySQL or other similar. A hosting package with preinstalled software and at least 1 GB for use data is also acceptable for both development and live deployment of IPOS.

8.2.2 optimal hardware configuration

The optimal hardware configuration envisaged for development should consist of a desktop PC, 4GB of RAM and at least 250GB of disk space for storing the software programs and data possibly running a database server such as PostgreSQL or better. Again, a hosting package with preinstalled software + 5GB of data is considered an optimal hardware configuration for both development and live deployment

9 Index

Not provided in this release of the document